

EcoBuck Backend Project Architecture Specification

FastAPI Enterprise-Grade Scalable Layout Blueprint

Author: Toqeer Ahmed
Role: Principal IoT Systems & Backend Architect
Date: July 8, 2026
Status: PROPOSED (Architecture Review Stage)

1. Directory Tree Overview

This blueprint describes a production-ready, clean, layered architecture for the **EcoBuck Backend** built using **FastAPI** and **Google Cloud Firestore**. The architecture isolates database operations, business logic, validation layers, and routes, and includes directories ready for future AI/ML models.

```
ecobuck-backend/  
├── .github/  
│   ├── workflows/  
│   └── ci-cd.yml  
├── app/  
│   ├── __init__.py  
│   ├── main.py  
│   └── api/  
│       ├── __init__.py  
│       ├── deps.py  
│       └── v1/  
│           ├── __init__.py  
│           ├── router.py  
│           └── endpoints/  
│               ├── auth.py  
│               ├── telemetry.py  
│               ├── devices.py  
│               └── alerts.py  
├── schemas/  
│   ├── __init__.py  
│   ├── auth.py  
│   ├── telemetry.py  
│   ├── devices.py  
│   └── alerts.py  
└── services/  
    ├── __init__.py  
    ├── auth.py  
    └── telemetry.py
```

```
■ ■ ■■■ devices.py
■ ■ ■■■ alerts.py
■ ■ ■■■ notification.py
■ ■■■ repositories/
■ ■ ■■■ __init__.py
■ ■ ■■■ base.py
■ ■ ■■■ user.py
■ ■ ■■■ device.py
■ ■ ■■■ telemetry.py
■ ■ ■■■ alert.py
■ ■■■ db/
■ ■ ■■■ __init__.py
■ ■ ■■■ firestore.py
■ ■ ■■■ base.py
■ ■■■ alert_engine/
■ ■ ■■■ __init__.py
■ ■ ■■■ rules.py
■ ■ ■■■ processor.py
■ ■■■ core/
■ ■ ■■■ __init__.py
■ ■ ■■■ config.py
■ ■ ■■■ security.py
■ ■ ■■■ exceptions.py
■ ■■■ ai/
■ ■ ■■■ __init__.py
■ ■ ■■■ predictor.py
■ ■ ■■■ anomaly.py
■ ■ ■■■ models/
■ ■ ■■■ .gitkeep
■ ■■■ utils/
■ ■■■ __init__.py
■ ■■■ logger.py
■ ■■■ time_utils.py
■■■ tests/
■ ■■■ __init__.py
■ ■■■ conftest.py
■ ■■■ api/
■ ■ ■■■ test_telemetry.py
■ ■■■ services/
■ ■ ■■■ test_alert_rules.py
■ ■■■ mock/
■ ■■■ test_db.py
■■■ simulator/
■ ■■■ __init__.py
■ ■■■ config.py
■ ■■■ run_simulation.py
■■■ deploy/
■ ■■■ Dockerfile
■ ■■■ docker-compose.yml
■ ■■■ k8s/
■ ■■■ deployment.yaml
■ ■■■ service.yaml
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

2. Directory & Module Responsibilities

2.1 Configuration & Base Settings

- `.env.example`
Responsibility: Template file documenting required environment variables (e.g. Firebase credentials, port settings, secret keys) for local developers without exposing secrets to git control.
- `app/core/config.py`
Responsibility: Implements a Pydantic-based configuration manager (`BaseSettings`) that validates environment variables at runtime, ensuring the server fails-fast if a critical credential (like Firebase credentials) is missing.
- `app/core/exceptions.py`
Responsibility: Defines custom domain exceptions (e.g., `DeviceAccessDeniedException`, `InvalidCompostStateException`) and maps them to FastAPI global HTTP exception handlers.

2.2 Routing & Web Layer (API)

- `app/main.py`
Responsibility: The application entrypoint. Initializes the FastAPI instance, registers global middlewares (CORS, Request Tracing), links API routers, and sets up database connections on startup/shutdown event lifecycles.
- `app/api/deps.py`
Responsibility: Declares global dependencies used in path parameters (e.g. `get_current_user`, `get_current_active_device`, `get_db`), enforcing OAuth2 JWT validation and RBAC (Role-Based Access Control) policies.
- `app/api/v1/router.py`
Responsibility: Assembles individual endpoint routers into a versioned namespace router (`/api/v1`).
- `app/api/v1/endpoints/auth.py`
Responsibility: Maps paths for user registration and JWT login/token-refresh operations.
- `app/api/v1/endpoints/telemetry.py`
Responsibility: Exposes the endpoint `/api/v1/telemetry` consumed by the ESP32 firmware to post raw and filtered sensor metrics.
- `app/api/v1/endpoints/devices.py`
Responsibility: Exposes device provisioning, device configuration updates, and manual validation controls (`/devices/{id}/validate`).

- `app/api/v1/endpoints/alerts.py`
Responsibility: Serves active warning retrieval and alert acknowledgment commands.
-

2.3 Data Validation (Schemas)

- `app/schemas/`
Responsibility: Contains Pydantic V2 models defining request and response JSON shapes, data conversion rules, and input constraints (e.g. checking email regex, validation of temperature intervals).
 - `app/schemas/auth.py`
Responsibility: Rules for login forms, token envelopes, and registration payloads.
 - `app/schemas/telemetry.py`
Responsibility: Validates telemetry payloads matching firmware models.
 - `app/schemas/devices.py`
Responsibility: Defines configuration updates and user-device claims mapping.
 - `app/schemas/alerts.py`
Responsibility: Validates request parameters and return logs for alerts.
-

2.4 Business Logic Layer (Services)

- `app/services/`
Responsibility: The core engine of the clean architecture. Services consume database repositories, execute business logic rules, and compose outputs. Controllers (API) should *never* query database drivers directly; they must call Services.
- `app/services/auth.py`
Responsibility: Handles password hashing, password validation, and token signing/validation wrappers.
- `app/services/telemetry.py`
Responsibility: Processes raw telemetry payloads, updates cache layers, and flags alerts to the rules engine.
- `app/services/devices.py`
Responsibility: Coordinates device pairing ownership logic and configurations changes.
- `app/services/alerts.py`
Responsibility: Resolves and audits warning states.
- `app/services/notification.py`
Responsibility: Integrates with Google Firebase Cloud Messaging (FCM) client to send background

push messages.

2.5 Security & Cryptography (Authentication)

- `app/core/security.py`
Responsibility: Implements low-level cryptography operations, password hashing configurations (using `bcrypt` or `argon2`), JWT signing keys, and HMAC verification for hardware device tokens.

2.6 Database Connectivity (DB & Repositories)

- `app/db/firestore.py`
Responsibility: Configures and initializes the singleton instance of the `firebase_admin` Google Cloud Firestore client.
- `app/db/base.py`
Responsibility: Setup connection pooling, context managers, and transaction execution scopes.
- `app/repositories/base.py`
Responsibility: Implements abstract generic NoSQL database patterns (CRUD interfaces) so the database client can be swapped easily (e.g., from Firestore to PostgreSQL) without rewriting business service code.
- `app/repositories/user.py` / `device.py` / `telemetry.py` / `alert.py`
Responsibility: Contains queries and commands mapping to specific Firestore collections (e.g. document lookups, array appends, subcollection range scans).

2.7 Rules & Anomaly Engine (Alert Engine)

- `app/alert_engine/processor.py`
Responsibility: Subscribes to telemetry events during ingestion, evaluating incoming data points against system rules.
- `app/alert_engine/rules.py`
Responsibility: Contains declarative criteria checking (e.g., checking if humidity is out of range, checking if device heartbeat has timed out).

2.8 Future AI / ML Engine (AI Scalability)

- `app/ai/predictor.py`
Responsibility: Entrypoint service that loads trained ML models (e.g., regression or LSTM models) to forecast compost maturity days and stability windows based on historical records retrieved from the telemetry repository.
 - `app/ai/anomaly.py`
Responsibility: ML-driven outlier and anomaly detection (e.g. predicting bad odors, classifying microbe degradation failures).
 - `app/ai/models/`
Responsibility: Storage directory for serialized model weights (like ONNX models, pickle files, or TensorFlow SavedModels).
-

2.9 Utility Functions

- `app/utils/logger.py`
Responsibility: Standardized logger configuration formatting server requests, hardware errors, and database transaction timings.
 - `app/utils/time_utils.py`
Responsibility: Timezone conversions (handling UTC offsets from field devices) and epoch parsing.
-

2.10 Automated Testing (Testing)

- `tests/conftest.py`
Responsibility: Setup Pytest fixtures, including a mock Firestore client instance to prevent test queries from writing to production environments.
 - `tests/api/test_telemetry.py`
Responsibility: Tests API endpoints using FastAPI `TestClient` (e.g. validating payload schema rejections).
 - `tests/services/test_alert_rules.py`
Responsibility: Validates state triggers and calculations within the rules engine.
 - `tests/mock/test_db.py`
Responsibility: Mocks transactional write/read behaviors.
-

2.11 Mock Hardware Suite (Simulator)

- `simulator/config.py`
Responsibility: Defines interval configurations, target server endpoints, and simulated device lists.
 - `simulator/run_simulation.py`
Responsibility: A standalone Python script that mimics physical ESP32 boards. It reads config, generates simulated temperature/humidity curves, and publishes JSON payloads to the `/telemetry` endpoint at intervals to support mobile app testers.
-

2.12 Infrastructure & Deployment

- `deploy/Dockerfile`
Responsibility: Multi-stage build Docker configuration that packages the FastAPI app, installs minimal dependencies, sets up a non-root security context, and runs Uvicorn.
 - `deploy/docker-compose.yml`
Responsibility: Orchestrates the FastAPI container, local environment setups, and secrets volumes.
 - `deploy/k8s/`
Responsibility: Kubernetes YAML configurations (`deployment.yaml`, `service.yaml`) for deploying, auto-scaling, and routing traffic to the backend services.
-

2.13 CI / CD Pipelines

- `.github/workflows/ci-cd.yml`
Responsibility: Defines GitHub Actions integration pipeline running `ruff/flake8` format checks, executing `pytest` test suites, building the Docker image, and uploading it to a container registry upon code merge.
-

2.14 Root Files

- `README.md`
Responsibility: Provides onboarding guides, API testing endpoints, project run parameters, environment configs, and simulator guidelines.
- `requirements.txt`
Responsibility: Core package dependencies (FastAPI, Uvicorn, Pydantic, Firebase Admin SDK, Cryptography).